

Projet RUST

Rapport de Projet

Abbruzzese Alexandre

Hengrui Ye

Alexandre Zhou-hu

Hulic-Menclé Nael

23 Janvier 2026

Table des matières

1	Introduction	3
2	Rôle et mission de chaque membre	3
3	Objectifs, planification et avancement	3
4	Difficultés, choix technologiques et algorithmiques	4
4.1	Pré-traitement et amélioration d'images (Alexandre Abbruzzese)	4
4.1.1	Représentation des données	4
4.1.2	Chargement et conversion	4
4.1.3	Transformations implémentées	4
4.1.4	Outil de dessin – Pinceau	5
4.1.5	Organisation du code	5
4.1.6	Problèmes rencontrés	6
4.1.7	Solutions apportées	6
4.1.8	Sources utilisées	6
4.2	Interface graphique (Hulic-Menclé Nael)	7
4.2.1	Choix technologiques	7
4.2.2	Organisation de l'état et de la fenêtre	7
4.2.3	Gestion des images et de la texture	8
4.2.4	Interaction utilisateur et déclenchement des traitements	8
4.2.5	Difficultés rencontrées et pistes d'amélioration	8
4.2.6	Fenêtre d'aide intégrée	9

4.3	Fonctionnalités avancées et architecture du logiciel	9
4.3.1	Gestion des interactions et interface graphique	9
4.3.2	Gestion des calques	10
4.3.3	Filtres et transformations	10
4.3.4	Outils de sélection	11
4.3.5	Historique des modifications (Undo / Redo)	11
4.3.6	Rendu et affichage final	12
4.4	Architecture / Projet (Alexandre Zhou-hu)	12
4.4.1	Rôle du module	12
4.4.2	Organisation du projet	12
4.4.3	Gestion technique et performances	12
4.4.4	Documentation d'installation	13
4.4.5	Problèmes rencontrés	13
4.4.6	Solutions apportées	13
5	Développement du Site Web du Projet	13
5.1	Choix Technologiques et Préparation	13
5.2	Inspirations et Références	14
5.3	Défis et Solutions	14
6	Conclusion	14

1 Introduction

Le projet porte sur la réalisation d'un module de traitement d'images, dont le but est de mettre en place différentes opérations de pré-traitement et de manipulation visuelle. Ce travail s'inscrit dans une approche orientée performance, fiabilité et clarté d'architecture, afin de garantir un développement propre, maintenable et efficace.

Le choix du langage Rust et de l'environnement Linux s'explique par leur robustesse, leur sécurité mémoire et leurs performances élevées, indispensables dans le cadre d'un traitement d'images de bas niveau. Le projet sera structuré sous la forme d'un package Cargo unique, permettant de centraliser tous les modules, bibliothèques et exécutables nécessaires à son bon fonctionnement.

Pour les opérations liées au traitement d'images, l'utilisation de crates spécialisées sera privilégiée, notamment `image`, afin de faciliter le chargement, la sauvegarde et le ré-encodage de divers formats tels que PNG ou JPEG, ainsi que la manipulation directe des pixels. Cette approche garantira une flexibilité suffisante pour intégrer plusieurs fonctionnalités de transformation et d'analyse.

L'ensemble du code respectera les règles de style établies, notamment une indentation cohérente, une limite de 80 caractères par ligne et l'absence de warnings à la compilation. Le projet adoptera une organisation modulaire claire, favorisant la lisibilité, la collaboration et l'évolution future du code. Enfin, la compilation et l'exécution seront assurées par l'outil Cargo, garantissant une intégration fluide et un déploiement fiable du système.

2 Rôle et mission de chaque membre

Alexandre Abbruzzese est responsable du module de pré-traitement et amélioration d'images ainsi que du développement du site web.

Hengrui Ye s'occupe des systèmes avancés (calques, sélection, undo/redo).

Hulic-Menclé Nael travaille sur l'interface graphique.

Alexandre Zhou-hu est responsable de l'architecture du projet.

3 Objectifs, planification et avancement

L'objectif principal est de développer une application de traitement d'image complète.

Le projet a été découpé en plusieurs étapes : conception, développement du moteur, fonctionnalités avancées, interface graphique et site web.

Actuellement, le moteur est fonctionnel, les fonctionnalités avancées sont partiellement implémentées et l'interface graphique est en cours.

4 Difficultés, choix technologiques et algorithmiques

4.1 Pré-traitement et amélioration d'images (Alexandre Abbruzese)

Cette partie du projet concerne le développement du module de pré-traitement et de manipulation d'images. Ce module constitue le cœur du traitement bas niveau, puisqu'il regroupe l'ensemble des opérations appliquées directement aux pixels.

L'objectif est de disposer d'une structure simple, efficace et réutilisable, permettant d'enchaîner plusieurs transformations sans dépendre des formats d'entrée.

4.1.1 Représentation des données

Le module repose sur deux structures principales :

Pixel : représente un pixel en RGBA

Image : contient les dimensions et un buffer linéaire de pixels

Le choix d'un buffer 1D permet de manipuler les données de manière plus directe, tout en gardant un bon contrôle sur la mémoire. L'accès à un pixel se fait via la formule classique :

$$index = y \times width + x$$

Cette approche évite la création de structures complexes et facilite l'implémentation des algorithmes.

Une structure Layer a également été introduite pour préparer l'ajout de calques, même si elle reste encore partiellement utilisée.

4.1.2 Chargement et conversion

Le chargement et la sauvegarde des images reposent sur la crate image. Cette bibliothèque est utilisée uniquement comme interface d'entrée/sortie.

Flux de traitement :

Chargement via image

Conversion vers la structure interne

Application des transformations

Conversion inverse pour la sauvegarde

4.1.3 Transformations implémentées

Les premières opérations mises en place sont des transformations simples :

niveaux de gris

inversion des couleurs

Détection des contours et flou de l'image
ajustement de luminosité
Resize et crop de l'image
Rotation de l'image
Ces traitements sont appliqués pixel par pixel :

```
for pixel in image {  
    pixel.r = 255 - pixel.r;  
    pixel.g = 255 - pixel.g;  
    pixel.b = 255 - pixel.b;  
}
```

4.1.4 Outil de dessin – Pinceau

Une nouvelle fonctionnalité a été intégrée au module : un outil pinceau permettant à l'utilisateur de dessiner directement sur les images. Cet outil s'inscrit dans la logique du projet en manipulant les pixels de l'image de manière interactive, à la manière d'un logiciel de retouche tel que Photoshop.

Fonctionnement L'outil pinceau permet d'appliquer une couleur choisie par l'utilisateur sur les pixels correspondant à la zone survolée par le curseur, selon une épaisseur définie. Le dessin est appliqué directement sur le buffer de l'image en temps réel lors du déplacement de la souris.

Paramètres exposés à l'utilisateur L'outil propose deux paramètres configurables :

- **Épaisseur du pinceau** : plusieurs niveaux d'épaisseur sont disponibles, permettant des traits fins pour les détails ou des traits épais pour remplir de grandes zones.
- **Couleur du pinceau** : l'utilisateur peut sélectionner librement la couleur souhaitée via un sélecteur de couleur, offrant ainsi une palette complète.

Implémentation L'application du pinceau repose sur la modification directe des pixels dans le buffer de l'image. Pour chaque position (x, y) touchée par le curseur, les pixels inclus dans le rayon défini par l'épaisseur sont mis à jour avec la couleur sélectionnée.

4.1.5 Organisation du code

Le module est séparé en plusieurs parties :
représentation des données
chargement / sauvegarde
transformations

Cette séparation permet de limiter les dépendances entre les composants et de rendre le code plus lisible.

4.1.6 Problèmes rencontrés

Plusieurs problèmes sont apparus lors du développement.

Problèmes de rendu :

Certaines transformations produisaient des images incorrectes (trop sombres, couleurs incohérentes). Cela venait principalement d'erreurs dans les calculs ou de dépassements de valeurs (ex : valeurs RGB hors $[0,255]$).

Gestion du buffer :

Des erreurs dans le calcul des indices ont parfois entraîné des décalages dans l'image ou des accès incorrects aux pixels. Ces bugs étaient difficiles à repérer car ils ne provoquaient pas toujours de crash, mais simplement un mauvais rendu.

Rust :

Quelques difficultés ont été rencontrées avec les règles d'emprunt, notamment lors de modifications en place du buffer. Certaines implémentations simples en théorie nécessitent d'être réécrites pour respecter les contraintes du langage.

4.1.7 Solutions apportées

Pour les problèmes de rendu, des tests ont été réalisés sur plusieurs images avec affichage intermédiaire des résultats. Cela a permis d'identifier rapidement les erreurs de calcul.

Une normalisation des valeurs a également été ajoutée :

clamp des valeurs RGB entre 0 et 255

vérification systématique après transformation

Concernant le buffer, des fonctions utilitaires ont été mises en place pour éviter les erreurs de calcul d'index :

```
fn getindex(x, y, width) -> usize y * width + x
```

Cela permet de centraliser la logique et d'éviter les erreurs répétées.

Pour Rust, les problèmes ont été corrigés en simplifiant certaines structures ou en utilisant des buffers temporaires lorsque nécessaire. L'utilisation de la documentation et d'exemples trouvés en ligne a permis de débloquer rapidement les cas les plus complexes.

4.1.8 Sources utilisées

Plusieurs ressources ont été utilisées pendant le développement :

documentation officielle de Rust

documentation de la crate image

vidéos YouTube pour certains concepts

forums (Stack Overflow) pour des problèmes spécifiques

Ces sources ont été utilisées principalement pour comprendre le fonctionnement du langage et résoudre des erreurs ponctuelles.

4.2 Interface graphique (Hulic-Menclé Nael)

Cette partie du projet correspond au module d'interface graphique. L'objectif est de proposer une fenêtre unique permettant de charger une image, d'appliquer les différents traitements du module `image_core` et de sauvegarder le résultat, sans avoir à taper de commandes ni manipuler des fichiers manuellement. La GUI joue ainsi le rôle de couche au-dessus du cœur de traitement : elle expose les filtres et transformations sous forme de menus, boutons, sliders et champs de texte, tout en affichant en permanence un aperçu de l'image courante.

4.2.1 Choix technologiques

Pour réaliser cette interface, nous choisissons d'utiliser Rust avec le duo `eframe/egui`. Ce framework permet de créer une application desktop native entièrement en Rust, évitant ainsi de mélanger plusieurs langages ou toolkits. `egui` fournit directement tous les éléments nécessaires : barres de menus, panneaux dockés, boutons, sliders, zones de texte, ainsi que l'affichage d'images sous forme de textures. Pour la sélection des fichiers, nous nous appuyons sur la crate `rfd`, qui ouvre les boîtes de dialogue natives du système et rend l'interface plus agréable à utiliser.

4.2.2 Organisation de l'état et de la fenêtre

Toute la logique de l'interface est regroupée dans une structure `App`, contenant l'ensemble de l'état nécessaire : chemins d'entrée et de sortie, image courante, copie originale, texture `egui`, message de statut et facteur de zoom. On y retrouve également tous les paramètres liés aux traitements (luminosité, contraste, flou, rotation, recadrage, redimensionnement, etc.) ainsi qu'un booléen `needs_texture_update` indiquant si la texture GPU doit être reconstruite.

La fenêtre est organisée en plusieurs zones distinctes :

- une barre de menus en haut,
- un `SidePanel` à gauche regroupant les contrôles (chargement, sauvegarde, actions générales, zoom, filtres, réglages, transformations),
- un `CentralPanel` dédié à l'affichage de l'image, indiquant sa résolution ou un message d'aide si aucune image n'est chargée.

4.2.3 Gestion des images et de la texture

Pour afficher une image dans `egui`, il faut la convertir de notre structure interne `Image` vers `egui::ColorImage`. Cette conversion est assurée par la fonction `image_to_color_image`, qui parcourt le buffer de pixels et transforme chaque `Pixel` en `Color32`.

Une fois la `ColorImage` obtenue, elle est chargée dans une texture via `ctx.load_texture`, ou mise à jour avec `set`. La fonction `sync_texture` centralise cette logique : elle ne re-construit la texture que lorsque `needs_texture_update` est vrai, évitant ainsi un travail inutile à chaque frame. L'image est ensuite affichée dans une zone scrollable, avec un facteur de zoom appliqué à la texture.

4.2.4 Interaction utilisateur et déclenchement des traitements

Toutes les interactions passent par des widgets `egui`. L'utilisateur peut saisir un chemin ou utiliser une boîte de dialogue pour sélectionner un fichier, puis charger ou sauvegarder l'image via les boutons dédiés.

Les filtres simples (*grayscale*, inversion, *sharpen*, Sobel) et les ajustements (luminosité, contraste, flou uniforme ou gaussien) sont déclenchés par des boutons. Pour éviter la duplication de code, tous ces traitements utilisent la fonction utilitaire `with_image_mut`, qui vérifie la présence d'une image, applique l'opération, met à jour la texture et modifie le message de statut.

Les transformations géométriques suivent le même principe : boutons pour les rotations prédéfinies, `DragValue` pour les angles arbitraires, recadrage et redimensionnement. Le choix du filtre de rééchantillonnage (*Nearest* ou *Bilinear*) permet d'observer les différences de rendu directement dans l'interface. Le zoom, quant à lui, n'agit que sur l'affichage.

4.2.5 Difficultés rencontrées et pistes d'amélioration

L'une des premières difficultés est de synchroniser correctement les données en mémoire et la texture affichée. Recharger la texture à chaque frame est simple mais coûteux, tandis qu'oublier de la mettre à jour après un filtre produit un rendu incohérent. L'introduction du champ `needs_texture_update` et la centralisation de la logique dans `sync_texture` permettent de résoudre ce problème.

Les performances représentent un autre défi : certains filtres étant lourds, les appliquer en continu lors du déplacement d'un slider rend l'interface peu fluide. C'est pourquoi la plupart des réglages sont associés à un bouton `Apply`.

Enfin, organiser tous les contrôles dans un seul panneau sans le rendre illisible nécessite un travail de structuration : sous-sections et séparateurs facilitent la navigation.

Dans l'ensemble, ce module d'interface graphique remplit pleinement son rôle : exposer les fonctionnalités du cœur de traitement dans une interface simple à prendre en main, tout en gérant les conversions, les erreurs et l'aperçu visuel. La base obtenue est suffisamment

solide pour accueillir des améliorations plus interactives (sélection à la souris, dessin direct, undo/redo visuel) sans remettre en cause les choix techniques initiaux.

4.2.6 Fenêtre d'aide intégrée

Une fenêtre d'aide a été intégrée à l'application afin d'accompagner l'utilisateur dans sa prise en main. Elle regroupe, dans une interface dédiée, l'ensemble des informations essentielles pour comprendre le fonctionnement du logiciel sans devoir consulter une documentation externe.

Cette aide présente de manière structurée les différentes étapes d'utilisation : chargement et sauvegarde d'images, fonctionnement des outils interactifs (sélection rectangulaire, pinceau), ainsi que le comportement associé à chacun d'eux. Les filtres et réglages disponibles y sont également décrits, avec une brève explication de leur effet visuel, permettant à l'utilisateur d'identifier rapidement le traitement adapté.

La fenêtre détaille aussi les transformations géométriques — rotations, recadrage, redimensionnement — ainsi que le rôle des filtres de rééchantillonnage. Enfin, une section récapitule les principaux raccourcis clavier, facilitant une utilisation plus fluide et efficace du logiciel.

Grâce à cette fonctionnalité, l'application offre un support clair et immédiatement accessible, améliorant l'ergonomie générale et rendant l'outil plus intuitif pour les nouveaux utilisateurs.

4.3 Fonctionnalités avancées et architecture du logiciel

Cette partie présente les systèmes avancés développés pour le logiciel de traitement d'image, notamment la gestion des calques, les filtres, les outils de sélection, le système d'historique ainsi que l'interface graphique.

4.3.1 Gestion des interactions et interface graphique

Dans ce projet, l'interface graphique joue un rôle central puisqu'elle permet de relier les actions de l'utilisateur aux différents modules internes du logiciel. L'interface a été développée avec les bibliothèques `egui` et `eframe`, qui permettent de créer une fenêtre interactive capable de gérer le clavier, la souris et l'affichage des images.

Le fonctionnement repose sur une boucle de mise à jour exécutée en continu. À chaque frame, le programme vérifie les événements utilisateur, comme les clics, les déplacements de souris ou les raccourcis clavier. Lorsqu'une action est détectée, le GUI appelle directement les fonctions correspondantes dans les modules du projet.

Par exemple, lorsqu'un utilisateur appuie sur `Ctrl + Z`, l'interface détecte cette combinaison grâce au contexte `ctx.input()` puis appelle la fonction `undo()` du système d'historique. Le même principe est utilisé pour `Ctrl + Y`, `Ctrl + C` et `Ctrl + V`.

L'interface permet également de relier les boutons graphiques aux fonctionnalités internes. Lorsqu'un bouton est cliqué, le GUI appelle directement les fonctions du **LayerManager**, du système de rendu ou des outils de traitement d'image. L'interface ne modifie jamais directement les pixels : elle sert uniquement d'intermédiaire entre l'utilisateur et la logique du programme.

4.3.2 Gestion des calques

Dans ce projet, un calque représente une image indépendante contenant ses propres pixels ainsi que plusieurs propriétés comme un identifiant, un nom et un état de visibilité. Chaque calque peut être modifié séparément sans affecter directement les autres couches de l'image.

La gestion des calques est assurée par une structure appelée **LayerManager**. Cette structure contient l'ensemble des calques du projet, le calque actuellement actif ainsi que la sélection courante. Elle centralise les opérations principales comme la création, la duplication, la suppression ou la modification de la visibilité des calques.

Lorsqu'un nouveau calque est créé, le programme lui attribue automatiquement un identifiant unique grâce à une variable `next_id`. Le calque est ensuite ajouté dans la liste des calques puis devient automatiquement le calque actif.

Chaque calque contient ses pixels sous forme d'un tableau `Vec<u8>` représentant des valeurs RGBA. Chaque pixel est composé de quatre octets correspondant au rouge, au vert, au bleu et à l'alpha. Cette organisation permet de manipuler directement les données de l'image.

Le système de visibilité permet également d'afficher ou de masquer certains calques. Lorsqu'un calque devient invisible, il n'est plus pris en compte lors du rendu final de l'image.

Le logiciel permet également la duplication des calques. Lorsqu'un calque est dupliqué, le programme crée une copie complète des données du calque original, lui attribue un nouvel identifiant ainsi qu'un nom modifié. Cette duplication permet de conserver une version originale tout en appliquant des transformations sur la copie.

4.3.3 Filtres et transformations

Le logiciel intègre plusieurs systèmes de transformation appliqués directement sur les pixels des images. Les filtres sont réalisés en parcourant les tableaux de pixels et en modifiant leurs composantes de couleur.

Le fonctionnement du filtre principal repose sur une inversion des couleurs. Le programme parcourt les pixels du calque actif et applique une formule de type :

$$\text{nouvelle_couleur} = 255 - \text{ancienne_couleur}$$

Cette opération est effectuée séparément sur les composantes rouge, verte et bleue de chaque pixel.

Avant d'appliquer un filtre, le programme vérifie si une sélection est active. Si c'est le cas, seuls les pixels appartenant à la sélection sont modifiés. Cette vérification est réalisée grâce à un masque de sélection contenant une valeur active pour chaque pixel sélectionné.

Les transformations sont directement reliées au système de calques. Les modifications sont appliquées uniquement sur le calque actif afin de préserver les autres couches de l'image.

4.3.4 Outils de sélection

Le logiciel intègre un système de sélection permettant de travailler sur une zone précise de l'image. La sélection est représentée par une structure contenant un masque binaire stocké dans un vecteur.

Chaque pixel de la sélection possède une valeur :

- 255 si le pixel est sélectionné ;
- 0 si le pixel ne l'est pas.

Lorsqu'un utilisateur clique et déplace la souris sur l'image, l'interface graphique détecte un événement de type **drag**. Les coordonnées de la souris sont ensuite converties dans le repère réel de l'image afin de créer correctement la zone sélectionnée.

Le programme génère ensuite un rectangle de sélection en parcourant les coordonnées comprises entre le point de départ et le point d'arrivée du déplacement de souris. Tous les pixels appartenant à cette zone reçoivent alors une valeur active dans le masque.

Cette sélection est utilisée par les filtres et les outils de dessin afin de limiter les modifications uniquement à certaines parties de l'image.

4.3.5 Historique des modifications (Undo / Redo)

Le logiciel possède un système d'historique permettant d'annuler ou de rétablir les actions effectuées par l'utilisateur.

Ce système repose sur une structure **History** contenant deux piles :

- **undo_stack**
- **redo_stack**

Avant chaque modification importante, le programme sauvegarde une action contenant l'état du calque avant et après la modification. Chaque action est représentée par une structure **Action**.

Lorsque l'utilisateur utilise **Ctrl + Z**, le programme récupère la dernière action présente dans **undo_stack**, restaure l'ancien état du calque puis place cette action dans **redo_stack**.

À l'inverse, `Ctrl + Y` récupère une action dans `redo_stack` afin de réappliquer la modification précédemment annulée.

Ce fonctionnement permet de conserver un historique cohérent des modifications tout en offrant une grande flexibilité à l'utilisateur pendant l'édition des images.

4.3.6 Rendu et affichage final

Le rendu final de l'image est assuré par un module spécifique chargé de fusionner tous les calques visibles du projet.

Le programme parcourt l'ensemble des calques contenus dans le `LayerManager`. Pour chaque calque visible, les pixels sont copiés dans une image finale qui sera ensuite affichée à l'écran.

Une fois l'image reconstruite, les données RGBA sont converties en texture graphique compatible avec `egui`. Cette texture est ensuite envoyée au GPU afin d'être affichée dans la fenêtre de l'application.

Le système de rendu est donc directement relié aux calques, aux outils de dessin, aux filtres et au système de sélection. Chaque modification effectuée dans le logiciel entraîne automatiquement une reconstruction de l'image finale affichée à l'utilisateur.

4.4 Architecture / Projet (Alexandre Zhou-hu)

4.4.1 Rôle du module

Cette partie du projet concerne l'organisation générale du logiciel ainsi que la manière dont les différents modules interagissent entre eux. Son rôle principal est de s'assurer que l'ensemble du logiciel est bien organisé, cohérent et fonctionne correctement dans son ensemble.

4.4.2 Organisation du projet

Le module définit comment le projet est découpé en différentes parties. Chaque partie du code a un rôle précis et clairement défini. Cette organisation permet aux développeurs de travailler plus facilement sur le projet et de comprendre rapidement son fonctionnement global.

Une interface claire est mise en place entre les modules afin de permettre leur communication sans dépendre de leur fonctionnement interne.

4.4.3 Gestion technique et performances

Une attention particulière est portée aux performances du programme afin d'éviter les ralentissements et garantir une bonne expérience utilisateur.

4.4.4 Documentation d'installation

Dans le cadre du projet, un fichier `installation.md` a été réalisé afin de faciliter la prise en main du logiciel par les utilisateurs et les développeurs. Cette documentation décrit de manière détaillée les différentes étapes nécessaires à l'installation du projet ainsi qu'à sa désinstallation.

Le fichier présente les prérequis nécessaires au bon fonctionnement de l'application, les commandes à exécuter pour installer les dépendances et lancer le projet, ainsi que les procédures permettant de supprimer proprement les fichiers et composants installés.

Cette documentation contribue à améliorer l'accessibilité et la maintenabilité du projet en fournissant un guide clair et structuré pour l'environnement d'installation.

4.4.5 Problèmes rencontrés

Des difficultés ont été rencontrées dans la communication entre les modules et dans l'organisation globale du projet.

4.4.6 Solutions apportées

Une organisation modulaire stricte a été mise en place pour résoudre ces problèmes et améliorer la lisibilité du code.

5 Développement du Site Web du Projet

En complément du développement du projet lui-même, nous avons également été responsables de la création du site web du projet. Ce site sert à présenter notre travail et à suivre l'évolution du développement.

5.1 Choix Technologiques et Préparation

Pour le développement du site web, nous avons opté pour :

- Vue.js comme framework JavaScript principal, grâce à notre familiarité antérieure avec cet outil et sa praticité
- Une stack classique de HTML, CSS et JavaScript pour la structure et les fonctionnalités de base
- Tailwind CSS pour le styling, apprécié pour son approche utilitaire permettant un développement rapide

Avant de commencer le développement, nous avons dû rafraîchir nos connaissances sur Vue.js en étudiant nos anciens projets et en consultant des tutoriels en ligne, car notre dernière utilisation du framework remontait à un certain temps.

5.2 Inspirations et Références

Pour le design du site, nous nous sommes inspirés de plusieurs sources :

- Le site de l’entreprise Anthropic pour son design épuré et fonctionnel, qui correspondait à l’image que nous souhaitions pour notre projet
- W3Schools pour des informations techniques précises sur les commandes HTML, CSS et JavaScript
- Des tutoriels YouTube pour comprendre certaines méthodes de développement spécifiques à Vue.js

Ces références nous ont permis de créer un site au design professionnel tout en répondant efficacement aux besoins du projet.

5.3 Défis et Solutions

Le développement du site web a présenté plusieurs défis :

Rafraîchissement des connaissances sur Vue.js

- Problème : Manque de maîtrise initiale du framework après une période sans l’utiliser
- Solution : Analyse de nos anciens projets et suivi de tutoriels en ligne pour actualiser nos compétences

Intégration de Tailwind CSS

- Problème : Difficultés initiales avec l’approche utilitaire de ce framework CSS
- Solution : Utilisation intensive de la documentation officielle et d’exemples pratiques

6 Conclusion

Ce projet a permis de développer une application de traitement d’images en Rust, combinant un moteur performant, une interface graphique et des fonctionnalités avancées comme les calques et l’undo/redo. Le module de pré-traitement est fonctionnel, les structures de données sont en place et les transformations principales opérationnelles, tandis que l’interface rend l’usage du logiciel simple et intuitif.

L’architecture modulaire assure une bonne organisation et facilite la maintenance, et le site web offre un support de présentation clair. Certaines limites subsistent, notamment dans l’enchaînement des filtres et la gestion des modifications successives, constituant des axes d’amélioration pour la suite.

Ainsi, ce projet constitue une base solide, évolutive et prête à être enrichie pour devenir un outil de traitement d’images complet et performant.